

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

Riya Gupta(1WA24CS235)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,

Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by Riya Gupta(1WA24CS235) who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026 . The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF (Preemptive and Non-Preemptive) c) Priority (Preemptive and Non-Preemptive) d) Round Robin (Experiment with different quantum sizes for RR algorithm)	1-20
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	21-24
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	25-32
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	33-38
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	39-44
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	45-47
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	49-53

Course Outcomes:

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

LAB1: FCFS — First Come First Serve (Non-Preemptive)

```
#include <stdio.h>

int main() {

    printf("Name: Riya Gupta\nUSN:1WA24CS235\n\n");

    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], at[n], bt[n], ct[n], tat[n], wt[n], rt[n];

    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("Enter AT and BT for P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    // --- Sort by Arrival Time (Bubble Sort) ---
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (at[j] > at[j + 1]) {
                int tmp;
                tmp = at[j];
                at[j] = at[j + 1];
                at[j + 1] = tmp;
                tmp = bt[j];
                bt[j] = bt[j + 1];
                bt[j + 1] = tmp;
                tmp = pid[j];
                pid[j] = pid[j + 1];
                pid[j + 1] = tmp;
            }

    // --- Completion Time ---
    // First process: starts at its own AT
    ct[0] = at[0] + bt[0];
    for (int i = 1; i < n; i++) {
        // If CPU is free before next process arrives, it waits
```

```

int start = (ct[i - 1] < at[i]) ? at[i] : ct[i - 1];
ct[i] = start + bt[i];
}

// --- TAT, WT, RT ---
float sumTAT = 0, sumWT = 0, sumRT = 0;
for (int i = 0; i < n; i++) {
    tat[i] = ct[i] - at[i]; // TAT = CT - AT
    wt[i] = tat[i] - bt[i]; // WT = TAT - BT
    rt[i] = wt[i];          // RT = WT (FCFS: no preemption)
    sumTAT += tat[i];
    sumWT += wt[i];
    sumRT += rt[i];
}

// --- Sort by PID before printing ---
for (int i = 0; i < n - 1; i++)
    for (int j = 0; j < n - i - 1; j++)
        if (pid[j] > pid[j + 1]) {
            int tmp;
            tmp = pid[j];
            pid[j] = pid[j + 1];
            pid[j + 1] = tmp;
            tmp = at[j];
            at[j] = at[j + 1];
            at[j + 1] = tmp;
            tmp = bt[j];
            bt[j] = bt[j + 1];
            bt[j + 1] = tmp;
            tmp = ct[j];
            ct[j] = ct[j + 1];
            ct[j + 1] = tmp;
            tmp = tat[j];
            tat[j] = tat[j + 1];
            tat[j + 1] = tmp;
            tmp = wt[j];
            wt[j] = wt[j + 1];
            wt[j + 1] = tmp;
            tmp = rt[j];
            rt[j] = rt[j + 1];
            rt[j + 1] = tmp;
        }
}

```

```

// --- Print Table ---
printf("\n%-6s %-6s %-6s %-6s %-6s %-6s %-6s\n", "PID", "AT", "BT", "CT",
    "TAT", "WT", "RT");
printf("-----\n");
for (int i = 0; i < n; i++)
    printf("P%-5d %-6d %-6d %-6d %-6d %-6d %-6d\n", pid[i], at[i], bt[i], ct[i],
        tat[i], wt[i], rt[i]);

printf("-----\n");
printf("Avg TAT: %.2f | Avg WT: %.2f | Avg RT: %.2f\n", sumTAT / n,
    sumWT / n, sumRT / n);

return 0;
}

```

Output:

```
riya gupta, 1WA24CS235Enter number of processes: 3
```

```
Process 1
```

```
Arrival Time: 2
```

```
Burst Time:
```

```
5
```

```
Process 2
```

```
Arrival Time: 3
```

```
Burst Time: 2
```

```
Process 3
```

```
Arrival Time: 4
```

```
Burst Time: 3
```

PID	AT	BT	CT	TAT	WT	RT
1	2	5	7	5	0	0
2	3	2	9	6	4	4
3	4	3	12	8	5	5

```
Average Turnaround Time = 6.33
```

```
Average Waiting Time = 3.00
```

```
Average Response Time = 3.00
```

```
=== Code Execution Successful ===
```

// SJF — Shortest Job First (Non-Preemptive)

```
#include <stdio.h>

int main() {

    printf("Name: Riya Gupta\nUSN:1WA24CS235\n\n");
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], at[n], bt[n], ct[n], tat[n], wt[n], rt[n], done[n];

    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("Enter AT and BT for P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        done[i] = 0;
    }

    // --- Simulate SJF ---
    int time = 0, completed = 0;
    while (completed < n) {

        // Find arrived, not-done process with minimum BT
        int idx = -1;
        for (int i = 0; i < n; i++) {
            if (!done[i] && at[i] <= time) {
                if (idx == -1 || bt[i] < bt[idx])
                    idx = i;
            }
        }

        if (idx == -1) {
            time++;
            continue;
        } // CPU idle, advance time

        rt[idx] = time - at[idx]; // RT = start time - AT
        time += bt[idx];          // process runs to completion
        ct[idx] = time;
        tat[idx] = ct[idx] - at[idx]; // TAT = CT - AT
        wt[idx] = tat[idx] - bt[idx]; // WT = TAT - BT
        done[idx] = 1;
        completed++;
    }

    // --- Sort by PID before printing ---
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (pid[j] > pid[j + 1]) {
                int tmp;
                tmp = pid[j];
            }
    }
```

```

    pid[j] = pid[j + 1];
    pid[j + 1] = tmp;
    tmp = at[j];
    at[j] = at[j + 1];
    at[j + 1] = tmp;
    tmp = bt[j];
    bt[j] = bt[j + 1];
    bt[j + 1] = tmp;
    tmp = ct[j];
    ct[j] = ct[j + 1];
    ct[j + 1] = tmp;
    tmp = tat[j];
    tat[j] = tat[j + 1];
    tat[j + 1] = tmp;
    tmp = wt[j];
    wt[j] = wt[j + 1];
    wt[j + 1] = tmp;
    tmp = rt[j];
    rt[j] = rt[j + 1];
    rt[j + 1] = tmp;
}

// --- Print Table ---
printf("\n%-6s %-6s %-6s %-6s %-6s %-6s %-6s\n", "PID", "AT", "BT", "CT",
        "TAT", "WT", "RT");
printf("-----\n");

float sumTAT = 0, sumWT = 0, sumRT = 0;
for (int i = 0; i < n; i++) {
    sumTAT += tat[i];
    sumWT += wt[i];
    sumRT += rt[i];
    printf("P%-5d %-6d %-6d %-6d %-6d %-6d %-6d\n", pid[i], at[i], bt[i], ct[i],
        tat[i], wt[i], rt[i]);
}

printf("-----\n");
printf("Avg TAT: %.2f | Avg WT: %.2f | Avg RT: %.2f\n", sumTAT / n,
    sumWT / n, sumRT / n);

return 0;
}

```

Output:

Output

```
riya gupta, 1WA24CS235Enter number of processes: 4
```

```
Process 1
```

```
Arrival Time: 3
```

```
Burst Time: 4
```

```
Process 2
```

```
Arrival Time: 2
```

```
Burst Time:
```

```
6
```

```
Process 3
```

```
Arrival Time: 1
```

```
Burst Time: 4
```

```
Process 4
```

```
Arrival Time: 3
```

```
Burst Time: 7
```

PID	AT	BT	CT	TAT	WT	RT
1	3	4	9	6	2	2
2	2	6	15	13	7	7
3	1	4	5	4	0	0
4	3	7	22	19	12	12

```
Average Turnaround Time = 10.50
```

```
Average Waiting Time = 5.25
```

```
Average Response Time = 5.25
```

SRTF — Shortest Remaining Time First (Preemptive SJF)

```
#include <stdio.h>

int main() {

    printf("Name: Riya Gupta\nUSN:1WA24CS235\n\n");

    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], at[n], bt[n], rem[n], ct[n], tat[n], wt[n], rt[n], done[n],
        started[n];

    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("Enter AT and BT for P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        rem[i] = bt[i]; // remaining time = burst time initially
        done[i] = 0;
        started[i] = 0;
    }

    // --- Simulate SRTF (tick by tick) ---
    int time = 0, completed = 0;
    while (completed < n) {

        // Find arrived, not-done process with minimum remaining time
        int idx = -1;
        for (int i = 0; i < n; i++) {
            if (!done[i] && at[i] <= time) {
                if (idx == -1 || rem[i] < rem[idx])
                    idx = i;
            }
        }

        if (idx == -1) {
            time++;
            continue;
        } // CPU idle

        // Capture RT the first time this process gets the CPU
        if (!started[idx]) {
            rt[idx] = time - at[idx]; // RT = first start - AT
            started[idx] = 1;
        }

        rem[idx]--; // run for 1 tick
        time++;

        // Process finished?
```

```

if (rem[idx] == 0) {
    ct[idx] = time;
    tat[idx] = ct[idx] - at[idx]; // TAT = CT - AT
    wt[idx] = tat[idx] - bt[idx]; // WT = TAT - BT
    done[idx] = 1;
    completed++;
}
}

// --- Sort by PID before printing ---
for (int i = 0; i < n - 1; i++)
    for (int j = 0; j < n - i - 1; j++)
        if (pid[j] > pid[j + 1]) {
            int tmp;
            tmp = pid[j];
            pid[j] = pid[j + 1];
            pid[j + 1] = tmp;
            tmp = at[j];
            at[j] = at[j + 1];
            at[j + 1] = tmp;
            tmp = bt[j];
            bt[j] = bt[j + 1];
            bt[j + 1] = tmp;
            tmp = ct[j];
            ct[j] = ct[j + 1];
            ct[j + 1] = tmp;
            tmp = tat[j];
            tat[j] = tat[j + 1];
            tat[j + 1] = tmp;
            tmp = wt[j];
            wt[j] = wt[j + 1];
            wt[j + 1] = tmp;
            tmp = rt[j];
            rt[j] = rt[j + 1];
            rt[j + 1] = tmp;
        }

// --- Print Table ---
printf("\n%-6s %-6s %-6s %-6s %-6s %-6s %-6s\n", "PID", "AT", "BT", "CT",
        "TAT", "WT", "RT");
printf("-----\n");

float sumTAT = 0, sumWT = 0, sumRT = 0;
for (int i = 0; i < n; i++) {
    sumTAT += tat[i];
    sumWT += wt[i];
    sumRT += rt[i];
    printf("P%-5d %-6d %-6d %-6d %-6d %-6d\n", pid[i], at[i], bt[i], ct[i],
        tat[i], wt[i], rt[i]);
}

printf("-----\n");
printf("Avg TAT: %.2f | Avg WT: %.2f | Avg RT: %.2f\n", sumTAT / n,

```

```
    sumWT / n, sumRT / n);  
  
    return 0;  
}
```

Output:

Output

```
riya gupta, 1WA24CS235Enter number of processes: 3
```

```
Process 1
```

```
Arrival Time: 2
```

```
Burst Time: 5
```

```
Process 2
```

```
Arrival Time: 3
```

```
Burst Time: 2
```

```
Process 3
```

```
Arrival Time: 1
```

```
Burst Time:
```

```
4
```

PID	AT	BT	CT	TAT	WT	RT
1	2	5	12	10	5	5
2	3	2	5	2	0	0
3	1	4	7	6	2	0

```
Average Turnaround Time = 6.00
```

```
Average Waiting Time = 2.33
```

```
Average Response Time = 1.67
```

```
=== Code Execution Successful ===
```

Priority Scheduling — Non-Preemptive

```
#include <stdio.h>

int main() {

    printf("Name: Riya Gupta\nUSN:1WA24CS235\n\n");
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], at[n], bt[n], pr[n], ct[n], tat[n], wt[n], rt[n], done[n];

    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("Enter AT, BT and PR for P%d: ", i + 1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        done[i] = 0;
    }

    // --- Simulate Priority (Non-Preemptive) ---
    int time = 0, completed = 0;
    while (completed < n) {

        // Find arrived, not-done process with minimum priority number
        int idx = -1;
        for (int i = 0; i < n; i++) {
            if (!done[i] && at[i] <= time) {
                if (idx == -1 || pr[i] < pr[idx])
                    idx = i;
            }
        }

        if (idx == -1) {
            time++;
            continue;
        } // CPU idle

        rt[idx] = time - at[idx]; // RT = start time - AT
        time += bt[idx];          // runs to completion
        ct[idx] = time;
        tat[idx] = ct[idx] - at[idx]; // TAT = CT - AT
        wt[idx] = tat[idx] - bt[idx]; // WT = TAT - BT
        done[idx] = 1;
        completed++;
    }

    // --- Sort by PID before printing ---
```

```

for (int i = 0; i < n - 1; i++)
    for (int j = 0; j < n - i - 1; j++)
        if (pid[j] > pid[j + 1]) {
            int tmp;
            tmp = pid[j];
            pid[j] = pid[j + 1];
            pid[j + 1] = tmp;
            tmp = at[j];
            at[j] = at[j + 1];
            at[j + 1] = tmp;
            tmp = bt[j];
            bt[j] = bt[j + 1];
            bt[j + 1] = tmp;
            tmp = pr[j];
            pr[j] = pr[j + 1];
            pr[j + 1] = tmp;
            tmp = ct[j];
            ct[j] = ct[j + 1];
            ct[j + 1] = tmp;
            tmp = tat[j];
            tat[j] = tat[j + 1];
            tat[j + 1] = tmp;
            tmp = wt[j];
            wt[j] = wt[j + 1];
            wt[j + 1] = tmp;
            tmp = rt[j];
            rt[j] = rt[j + 1];
            rt[j + 1] = tmp;
        }

// --- Print Table ---
printf("\n%-6s %-6s %-6s %-6s %-6s %-6s %-6s %-6s\n", "PID", "AT", "BT", "PR",
        "CT", "TAT", "WT", "RT");
printf("-----\n");

float sumTAT = 0, sumWT = 0, sumRT = 0;
for (int i = 0; i < n; i++) {
    sumTAT += tat[i];
    sumWT += wt[i];
    sumRT += rt[i];
    printf("P%-5d %-6d %-6d %-6d %-6d %-6d %-6d %-6d\n", pid[i], at[i], bt[i],
        pr[i], ct[i], tat[i], wt[i], rt[i]);
}

printf("-----\n");
printf("Avg TAT: %.2f | Avg WT: %.2f | Avg RT: %.2f\n", sumTAT / n,
        sumWT / n, sumRT / n);
return 0;

```

output:

Output

```
riya gupta ,usn:1WA24CS235
Enter number of processes: 5

Enter details for Process 1
Arrival Time: 0
Burst Time: 3
Priority (Lower number = Higher priority): 5

Enter details for Process 2
Arrival Time: 2
Burst Time: 2
Priority (Lower number = Higher priority): 3

Enter details for Process 3
Arrival Time: 3
Burst Time: 5
Priority (Lower number = Higher priority): 2

Enter details for Process 4
Arrival Time: 4
Burst Time: 4
Priority (Lower number = Higher priority): 4

Enter details for Process 5
Arrival Time: 6
Burst Time: 1
Priority (Lower number = Higher priority): 1

PID AT  BT  PR  CT  TAT WT
P1  0   3   5   3   3   0
P2  2   2   3  11   9   7
P3  3   5   2   8   5   0
P4  4   4   4  15  11   7
P5  6   1   1   9   3   2
```

// Priority Scheduling — Preemptive

```
#include <stdio.h>

int main() {

    printf("Name: Riya Gupta\nUSN:1WA24CS235\n\n");
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], at[n], bt[n], pr[n], rem[n], ct[n], tat[n], wt[n], rt[n], done[n],
        started[n];

    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("Enter AT, BT and PR for P%d: ", i + 1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        rem[i] = bt[i];
        done[i] = 0;
        started[i] = 0;
    }

    // --- Simulate Priority (Preemptive, tick by tick) ---
    int time = 0, completed = 0;
    while (completed < n) {

        // Find arrived, not-done process with minimum priority number
        int idx = -1;
        for (int i = 0; i < n; i++) {
            if (!done[i] && at[i] <= time) {
                if (idx == -1 || pr[i] < pr[idx])
                    idx = i;
            }
        }

        if (idx == -1) {
            time++;
            continue;
        } // CPU idle

        // Capture RT the first time this process gets the CPU
        if (!started[idx]) {
            rt[idx] = time - at[idx]; // RT = first start - AT
            started[idx] = 1;
        }

        rem[idx]--; // run for 1 tick
        time++;

        // Process finished?
        if (rem[idx] == 0) {
            ct[idx] = time;
```

```

    tat[idx] = ct[idx] - at[idx]; // TAT = CT - AT
    wt[idx] = tat[idx] - bt[idx]; // WT = TAT - BT
    done[idx] = 1;
    completed++;
}
}

// --- Sort by PID before printing ---
for (int i = 0; i < n - 1; i++)
    for (int j = 0; j < n - i - 1; j++)
        if (pid[j] > pid[j + 1]) {
            int tmp;
            tmp = pid[j];
            pid[j] = pid[j + 1];
            pid[j + 1] = tmp;
            tmp = at[j];
            at[j] = at[j + 1];
            at[j + 1] = tmp;
            tmp = bt[j];
            bt[j] = bt[j + 1];
            bt[j + 1] = tmp;
            tmp = pr[j];
            pr[j] = pr[j + 1];
            pr[j + 1] = tmp;
            tmp = ct[j];
            ct[j] = ct[j + 1];
            ct[j + 1] = tmp;
            tmp = tat[j];
            tat[j] = tat[j + 1];
            tat[j + 1] = tmp;
            tmp = wt[j];
            wt[j] = wt[j + 1];
            wt[j + 1] = tmp;
            tmp = rt[j];
            rt[j] = rt[j + 1];
            rt[j + 1] = tmp;
        }

// --- Print Table ---
printf("\n%-6s %-6s %-6s %-6s %-6s %-6s %-6s %-6s\n", "PID", "AT", "BT", "PR",
        "CT", "TAT", "WT", "RT");
printf("-----\n");

float sumTAT = 0, sumWT = 0, sumRT = 0;
for (int i = 0; i < n; i++) {
    sumTAT += tat[i];
    sumWT += wt[i];
    sumRT += rt[i];
    printf("P%-5d %-6d %-6d %-6d %-6d %-6d %-6d\n", pid[i], at[i], bt[i],
        pr[i], ct[i], tat[i], wt[i], rt[i]);
}

printf("-----\n");

```

```
printf("Avg TAT: %.2f | Avg WT: %.2f | Avg RT: %.2f\n", sumTAT / n,  
      sumWT / n, sumRT / n);  
  
return 0;  
}
```

Output:

```
riya gupta ,usn:1WA24CS235  
Enter number of processes: 7  
  
Process 1  
Process ID: 6  
Arrival Time:  
6  
Burst Time: 5  
Priority: 6  
  
Process 2  
Process ID: 4  
Arrival Time: 4  
Burst Time: 1  
Priority: 5  
  
Process 3  
Process ID: 2  
Arrival Time: 1  
Burst Time: 2  
Priority: 4  
  
Process 4  
Process ID: 3  
Arrival Time: 3  
Burst Time: 4  
Priority: 4  
  
Process 5  
Process ID: 1  
Arrival Time: 0  
Burst Time: 8  
Priority: 3
```

// Round Robin — Preemptive

```
#include <stdio.h>

int main() {

    printf("Name: Riya Gupta\nUSN:1WA24CS235\n\n");
    int n, tq;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], at[n], bt[n], rem[n], ct[n], tat[n], wt[n], rt[n], started[n],
        inQueue[n];

    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("Enter AT and BT for P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        started[i] = 0;
        inQueue[i] = 0;
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    // --- Sort by Arrival Time ---
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (at[j] > at[j + 1]) {
                int tmp;
                tmp = at[j];
                at[j] = at[j + 1];
                at[j + 1] = tmp;
                tmp = bt[j];
                bt[j] = bt[j + 1];
                bt[j + 1] = tmp;
                tmp = pid[j];
                pid[j] = pid[j + 1];
                pid[j + 1] = tmp;
            }

    // Initialize rem after sort
    for (int i = 0; i < n; i++)
        rem[i] = bt[i];

    // --- Queue Setup ---
    int queue[1000], front = 0, rear = 0;

    int time = 0, completed = 0;

    // Start time at first arrival, seed queue with all processes arriving then
    time = at[0];
    for (int i = 0; i < n; i++)
```

```

if (at[i] <= time) {
    queue[rear++] = i;
    inQueue[i] = 1;
}

// --- Simulate Round Robin ---
while (completed < n) {

    if (front == rear) { // queue empty, CPU idle
        time++;
        for (int i = 0; i < n; i++) // check for new arrivals
            if (!inQueue[i] && rem[i] > 0 && at[i] <= time) {
                queue[rear++] = i;
                inQueue[i] = 1;
            }
        continue;
    }

    int idx = queue[front++]; // dequeue next process

    // Capture RT on first execution
    if (!started[idx]) {
        rt[idx] = time - at[idx]; // RT = first start - AT
        started[idx] = 1;
    }

    int exec = (rem[idx] < tq) ? rem[idx] : tq; // run for min(rem, tq)
    rem[idx] -= exec;
    time += exec;

    // Add newly arrived processes after this quantum
    for (int i = 0; i < n; i++)
        if (!inQueue[i] && rem[i] > 0 && at[i] <= time) {
            queue[rear++] = i;
            inQueue[i] = 1;
        }

    if (rem[idx] == 0) { // process finished
        ct[idx] = time;
        tat[idx] = ct[idx] - at[idx]; // TAT = CT - AT
        wt[idx] = tat[idx] - bt[idx]; // WT = TAT - BT
        completed++;
    } else {
        queue[rear++] = idx; // re-add to back of queue
    }
}

// --- Sort by PID before printing ---
for (int i = 0; i < n - 1; i++)
    for (int j = 0; j < n - i - 1; j++)
        if (pid[j] > pid[j + 1]) {
            int tmp;
            tmp = pid[j];

```

```

    pid[j] = pid[j + 1];
    pid[j + 1] = tmp;
    tmp = at[j];
    at[j] = at[j + 1];
    at[j + 1] = tmp;
    tmp = bt[j];
    bt[j] = bt[j + 1];
    bt[j + 1] = tmp;
    tmp = ct[j];
    ct[j] = ct[j + 1];
    ct[j + 1] = tmp;
    tmp = tat[j];
    tat[j] = tat[j + 1];
    tat[j + 1] = tmp;
    tmp = wt[j];
    wt[j] = wt[j + 1];
    wt[j + 1] = tmp;
    tmp = rt[j];
    rt[j] = rt[j + 1];
    rt[j + 1] = tmp;
}

// --- Print Table ---
printf("\n%-6s %-6s %-6s %-6s %-6s %-6s %-6s\n", "PID", "AT", "BT", "CT",
        "TAT", "WT", "RT");
printf("-----\n");

float sumTAT = 0, sumWT = 0, sumRT = 0;
for (int i = 0; i < n; i++) {
    sumTAT += tat[i];
    sumWT += wt[i];
    sumRT += rt[i];
    printf("P%-5d %-6d %-6d %-6d %-6d %-6d %-6d\n", pid[i], at[i], bt[i], ct[i],
        tat[i], wt[i], rt[i]);
}

return 0;
}

```

Output:

```
riya gupta ,usn:1WA24CS235
Enter number of processes: 5
Enter Arrival Time and Burst Time for P1: 0
5
Enter Arrival Time and Burst Time for P2: 1 3
Enter Arrival Time and Burst Time for P3: 2 1
Enter Arrival Time and Burst Time for P4: 3 2
Enter Arrival Time and Burst Time for P5: 4 3
Enter Time Quantum: 2
```

PID	AT	BT	WT	TAT	RT
P1	0	5	9	14	0
P2	1	3	8	11	1
P3	2	1	2	3	2
P4	3	2	2	4	2
P5	4	3	6	9	3

```
Average Waiting Time = 5.40
Average Turnaround Time = 8.20
Average Response Time = 1.60
```

```
=== Code Execution Successful ===
```

LAB 2:

MULTI-QUEUE:

```
#include <stdio.h>

int main() {
    printf("Name: Riya Gupta\nUSN:1WA24CS235\n\n");
    int n, curr_time = 0, i, j;
    int ns = 0, nu = 0;
    int type[10], tq;

    int completedSys = 0, completedUsr = 0;

    int q[10], f = 0, r = -1;
    int inq[10] = {0}, remt[10];

    int sysPID[10], sysAT[10], sysBT[10], sysCT[10], sysTAT[10], sysWT[10];
    int usrPID[10], usrAT[10], usrBT[10], usrCT[10], usrTAT[10], usrWT[10];

    double tat_sum = 0, wt_sum = 0;

    printf("Enter no. of processes: ");
    scanf("%d", &n);

    // input
    for (i = 0; i < n; i++) {

        printf("Enter type for process[%d] (0=system, 1=user): ", i + 1);
        scanf("%d", &type[i]);

        printf("Enter AT and BT for process[%d]: ", i + 1);

        if (type[i] == 0) {
            scanf("%d %d", &sysAT[ns], &sysBT[ns]);
            sysPID[ns] = i + 1;
            remt[ns] = sysBT[ns];
            ns++;
        } else {
            scanf("%d %d", &usrAT[nu], &usrBT[nu]);
            usrPID[nu] = i + 1;
            nu++;
        }
    }

    printf("Enter time quantum: ");
    scanf("%d", &tq);

    // run until both queues finished
    while (completedSys < ns || completedUsr < nu) {

        // add system processes to RR queue
        for (i = 0; i < ns; i++) {
            if (sysAT[i] <= curr_time && remt[i] > 0 && !inq[i]) {
```

```

    r = (r + 1) % 10;
    q[r] = i;
    inq[i] = 1;
}
}

// ----- check if system queue empty -----
if (f == (r + 1) % 10) {

    int ranUser = 0;

    // run user FCFS if ready
    for (i = 0; i < nu; i++) {

        if (usrBT[i] > 0 && usrAT[i] <= curr_time) {

            curr_time += usrBT[i];

            usrCT[i] = curr_time;

            usrTAT[i] = usrCT[i] - usrAT[i];
            usrWT[i] = usrTAT[i] - usrBT[i];

            tat_sum += usrTAT[i];
            wt_sum += usrWT[i];

            usrBT[i] = 0;
            completedUsr++;

            ranUser = 1;
            break;
        }
    }

    if (!ranUser)
        curr_time++;

    continue;
}

// ----- run system RR -----
i = q[f];
f = (f + 1) % 10;
inq[i] = 0;

if (f == (r + 1) % 10) {
    f = 0;
    r = -1;
}

if (remt[i] < tq) {
    curr_time += remt[i];
    remt[i] = 0;
}

```

```

} else {
    curr_time += tq;
    remt[i] -= tq;
}

// add new arrivals
for (j = 0; j < ns; j++) {
    if (sysAT[j] <= curr_time && remt[j] > 0 && !inq[j]) {
        r = (r + 1) % 10;
        q[r] = j;
        inq[j] = 1;
    }
}

// finished
if (remt[i] == 0) {

    sysCT[i] = curr_time;

    sysTAT[i] = sysCT[i] - sysAT[i];
    sysWT[i] = sysTAT[i] - sysBT[i];

    tat_sum += sysTAT[i];
    wt_sum += sysWT[i];

    completedSys++;

} else {

    r = (r + 1) % 10;
    q[r] = i;
    inq[i] = 1;
}

// print
printf("\nPID\tAT\tBT\tType\tCT\tTAT\tWT\n\n");

for (j = 1; j <= n; j++) {

    for (i = 0; i < ns; i++) {
        if (sysPID[i] == j) {
            printf("P%d\t%d\t%d\tSystem\t%d\t%d\t%d\n", sysPID[i], sysAT[i],
                sysBT[i], sysCT[i], sysTAT[i], sysWT[i]);
        }
    }

    for (i = 0; i < nu; i++) {
        if (usrPID[i] == j) {
            printf("P%d\t%d\t%d\tUser\t%d\t%d\t%d\n", usrPID[i], usrAT[i], usrBT[i],
                usrCT[i], usrTAT[i], usrWT[i]);
        }
    }
}

```

```

}

printf("\nAverage TAT: %.2f\n", tat_sum / n);
printf("Average WT: %.2f\n", wt_sum / n);

return 0;
}

```

Output

```

riya gupta , usn:1WA24CS235
Enter number of processes: 4

```

```

Process 1
Arrival Time: 0
Burst Time: 4
Type (1=System,2=User): 1

```

```

Process 2
Arrival Time: 0
Burst Time: 3
Type (1=System,2=User): 1

```

```

Process 3
Arrival Time: 0
Burst Time: 8
Type (1=System,2=User): 2

```

```

Process 4
Arrival Time: 10
Burst Time: 5
Type (1=System,2=User): 1

```

```

---User Queue---

```

PID	AT	BT	CT	TAT	WT
3	0	8	23	23	15

```

Average Turnaround Time=9.75
Average Waiting Time=4.75

```

LAB 3:

// Rate Monotonic

```
#include <stdio.h>

struct Task {
    int id;
    int execution;
    int period;
    int remaining;
    int next_release;
    int priority;
};

int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

int lcm(int a, int b) { return (a * b) / gcd(a, b); }

int main() {

    printf("Name: Riya Gupta\nUSN:1WA24CS235\n\n");

    int n, i, t, hyper = 1;

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    struct Task task[n];

    for (i = 0; i < n; i++) {
        printf("\nTask %d Execution Time: ", i + 1);
        scanf("%d", &task[i].execution);

        printf("Task %d Period: ", i + 1);
        scanf("%d", &task[i].period);

        task[i].id = i + 1;
        task[i].remaining = 0;
        task[i].next_release = 0;
    }

    // Assign priority (smaller period = higher priority)
```

```

for (i = 0; i < n; i++) {
    task[i].priority = task[i].period;
}

// Calculate hyperperiod (LCM of all periods)
for (i = 0; i < n; i++) {
    hyper = lcm(hyper, task[i].period);
}

printf("\nHyperperiod = %d\n", hyper);

printf("\nTime\tRunning Task\n");

for (t = 0; t < hyper; t++) {
    // Release tasks
    for (i = 0; i < n; i++) {
        if (t == task[i].next_release) {
            task[i].remaining = task[i].execution;
            task[i].next_release += task[i].period;
        }
    }

    int best = -1;

    // Choose highest priority task (smallest period)
    for (i = 0; i < n; i++) {
        if (task[i].remaining > 0) {
            if (best == -1 || task[i].priority < task[best].priority) {
                best = i;
            }
        }
    }

    if (best != -1) {
        printf("%d\tT%d\n", t, task[best].id);
        task[best].remaining--;
    } else {
        printf("%d\tIdle\n", t);
    }
}

return 0;
}

```

Output

```
riya gupta,USN 1WA24CS235
Enter the number of processes:2
Enter the CPU burst times:
20
35
Enter the time periods:
50
100
LCM=100

Rate Monotone Scheduling:
PID Burst   Period
1    20    50
2    35   100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle

=== Code Execution Successful ===
```

// Earliest Deadline First

```
#include <stdio.h>

struct Task {
    int id;
    int execution;
    int period;
    int deadline; // relative deadline
    int remaining;
    int next_release;
    int abs_deadline; // absolute deadline
};

int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

int lcm(int a, int b) { return (a * b) / gcd(a, b); }

int main() {

    int n, i, t, hyper = 1;

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    struct Task task[n];

    for (i = 0; i < n; i++) {
        task[i].id = i + 1;

        printf("\nTask %d Execution Time: ", i + 1);
        scanf("%d", &task[i].execution);

        printf("Task %d Period: ", i + 1);
        scanf("%d", &task[i].period);

        printf("Task %d Deadline: ", i + 1);
        scanf("%d", &task[i].deadline);

        task[i].remaining = 0;
        task[i].next_release = 0;
        task[i].abs_deadline = 0;
    }
}
```

```

// Hyperperiod
for (i = 0; i < n; i++) {
    hyper = lcm(hyper, task[i].period);
}

printf("\nHyperperiod = %d\n", hyper);
printf("\nTime\tRunning Task\n");

for (t = 0; t < hyper; t++) {
    // Release tasks
    for (i = 0; i < n; i++) {
        if (t == task[i].next_release) {
            task[i].remaining = task[i].execution;
            task[i].abs_deadline = t + task[i].deadline;
            task[i].next_release += task[i].period;
        }
    }

    int best = -1;

    // Choose earliest deadline
    for (i = 0; i < n; i++) {
        if (task[i].remaining > 0) {
            if (best == -1 || task[i].abs_deadline < task[best].abs_deadline) {
                best = i;
            }
        }
    }

    if (best != -1) {
        printf("%d\tT%d\n", t, task[best].id);
        task[best].remaining--;
    } else {
        printf("%d\tIdle\n", t);
    }
}

return 0;
}

```

Output:

Output

```
riya gupta,usn:1WA24CS235
Enter number of tasks: 3
Task 1 (burst deadline period): 2 4 5
Task 2 (burst deadline period): 3 7 20
Task 3 (burst deadline period): 2 8 10
```

PID	Burst	Deadline	Period
1	2	4	5
2	3	7	20
3	2	8	10

Scheduling occurs for 20 ms

```
0ms : Task 1 is running.
1ms : Task 1 is running.
2ms : Task 2 is running.
3ms : Task 2 is running.
4ms : Task 2 is running.
5ms : Task 1 is running.
6ms : Task 1 is running.s
7ms : Task 3 is running.
8ms : Task 3 is running.
9ms : CPU is idle.
10ms : Task 1 is running.
11ms : Task 1 is running.
12ms : Task 3 is running.
13ms : Task 3 is running.
14ms : CPU is idle.
15ms : Task 1 is running.
16ms : Task 1 is running.
17ms : CPU is idle.
18ms : CPU is idle.
19ms : CPU is idle.
```

```
=== Code Execution Successful ===
```

Proportional scheduling

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct {
    char name[5];
    int tickets;
} Process;

int main() {
    int n, total_tickets = 0;
    float total_T = 0.0;

    printf("Enter the number of Processes: ");
    scanf("%d", &n);
    Process p[n];
    srand(time(NULL));

    for (int i = 0; i < n; i++) {
        printf("\nProcess %d:\n", i + 1);
        sprintf(p[i].name, "P%d", i + 1);
        printf("Tickets: ");
        scanf("%d", &p[i].tickets);
        total_tickets += p[i].tickets;
    }

    printf("\n--- Proportional Share Scheduling ---\n");
    printf("Enter the Time Period for scheduling: ");
    int m;
    scanf("%d", &m);

    for (int i = 0; i < m; i++) {
        int winning_ticket = rand() % total_tickets + 1;
        int accumulated_tickets = 0;
        int winner_index;

        for (int j = 0; j < n; j++) {
            accumulated_tickets += p[j].tickets;
            if (winning_ticket <= accumulated_tickets) {
                winner_index = j;
                break;
            }
        }
        for (int i = 0; i < n; i++) {
            printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
                ((p[i].tickets / total_T) * 100))
        }
        return 0;
    }
}
```

Output:

Output

```
usn:1WA24CS235
name:Riya gupta
Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 18, Winner: P2
Tickets picked: 9, Winner: P1
Tickets picked: 10, Winner: P1
Tickets picked: 51, Winner: P3
Tickets picked: 51, Winner: P3

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.
```

LAB 4:

Producer consumer problem using semaphores

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty;      // counts empty slots
sem_t full;        // counts filled slots
pthread_mutex_t mutex; // for mutual exclusion

// Producer function
void *producer(void *arg) {
    int item, i = 0;

    while (1) {
        item = i++; // produce an item

        sem_wait(&empty);      // wait for empty slot
        pthread_mutex_lock(&mutex); // enter critical section

        buffer[in] = item;
        printf("Produced: %d at buffer[%d]\n", item, in);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex); // leave critical section
        sem_post(&full);              // signal item available

        sleep(1); // simulate production time
    }
}

// Consumer function
void *consumer(void *arg) {
    int item;

    while (1) {
        sem_wait(&full);      // wait for filled slot
        pthread_mutex_lock(&mutex); // enter critical section

        item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out);

        out = (out + 1) % BUFFER_SIZE;
```

```

pthread_mutex_unlock(&mutex); // leave critical section
sem_post(&empty);           // signal empty slot

sleep(2); // simulate consumption time
}
}

int main() {

pthread_t prod, cons;

sem_init(&empty, 0, BUFFER_SIZE);
sem_init(&full, 0, 0);
pthread_mutex_init(&mutex, NULL);

pthread_create(&prod, NULL, producer, NULL);
pthread_create(&cons, NULL, consumer, NULL);

pthread_join(prod, NULL);
pthread_join(cons, NULL);

sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

return 0;
}

```

Output:

Output

Name: Riya Gupta

USN:1WA24CS235

Produced: 0 at buffer[0]

Consumed: 0 from buffer[0]

Produced: 1 at buffer[1]

Consumed: 1 from buffer[1]

Produced: 2 at buffer[2]

Produced: 3 at buffer[3]

Consumed: 2 from buffer[2]

Produced: 4 at buffer[4]

Produced: 5 at buffer[0]

Consumed: 3 from buffer[3]

Produced: 6 at buffer[1]

Produced: 7 at buffer[2]

Consumed: 4 from buffer[4]

Produced: 8 at buffer[3]

Produced: 9 at buffer[4]

Consumed: 5 from buffer[0]

Produced: 10 at buffer[0]

Consumed: 6 from buffer[1]

Produced: 11 at buffer[1]

Consumed: 7 from buffer[2]

// Dining-Philosopher's problem

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

#define N 5 // Number of philosophers

pthread_mutex_t forks[N]; // One mutex per fork
pthread_t philosophers[N];

void *philosopher(void *num) {
    int id = *(int *)num;
    int left = id;          // left fork index
    int right = (id + 1) % N; // right fork index

    while (1) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1); // Thinking

        // To avoid deadlock, philosophers with even id pick left then right,
        // odd id pick right then left.
        if (id % 2 == 0) {
            // Pick up left fork
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);

            // Pick up right fork
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);
        } else {
            // Pick up right fork
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);

            // Pick up left fork
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
        }

        // Eating
        printf("Philosopher %d is eating.\n", id);
        sleep(2);

        // Put down forks
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);

        printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
    }
}
```

```

    return NULL;
}

int main() {

    int i;
    int ids[N];

    // Initialize forks (mutexes)
    for (i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }

    // Create philosopher threads
    for (i = 0; i < N; i++) {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    // Join threads (will never happen here, but good practice)
    for (i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }

    // Destroy mutexes (unreachable here)
    for (i = 0; i < N; i++) {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}

```

Output:

Name: Riya Gupta

USN: 1WA24CS235

```
Philosopher 0 is thinking.  
Philosopher 1 is thinking.  
Philosopher 2 is thinking.  
Philosopher 3 is thinking.  
Philosopher 4 is thinking.  
Philosopher 0 picked up left fork 0.  
Philosopher 0 picked up right fork 1.  
Philosopher 0 is eating.  
Philosopher 1 picked up right fork 2.  
Philosopher 3 picked up right fork 4.  
Philosopher 3 picked up left fork 3.  
Philosopher 3 is eating.  
Philosopher 0 put down forks 0 and 1.  
Philosopher 0 is thinking.  
Philosopher 1 picked up left fork 1.  
Philosopher 1 is eating.  
Philosopher 3 put down forks 3 and 4.  
Philosopher 3 is thinking.  
Philosopher 4 picked up left fork 4.  
Philosopher 4 picked up right fork 0.  
Philosopher 4 is eating.  
Philosopher 1 put down forks 1 and 2.  
Philosopher 1 is thinking.  
Philosopher 2 picked up left fork 2.  
Philosopher 2 picked up right fork 3.
```

LAB 5:

Deadlock Avoidance using Banker's Algorithm

```
#include <stdio.h>

int main() {
    int n, m;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], avail[m];
    int need[n][m], finish[n], safeSeq[n];

    // Input Allocation Matrix
    printf("Enter Allocation Matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    // Input Maximum Demand Matrix
    printf("Enter Maximum Demand Matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    // Input Available Resources
    printf("Enter Available Resources:\n");
    for (int j = 0; j < m; j++)
        scanf("%d", &avail[j]);

    // Step 1: Calculate Need and initialize Finish
    for (int i = 0; i < n; i++) {
        finish[i] = 0; // false
        for (int j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
    }

    // Work = Available
    int work[m];
    for (int j = 0; j < m; j++)
        work[j] = avail[j];

    // Step 2 & 3: Safety Algorithm
    int count = 0;
    while (count < n) {
        int found = 0;
        for (int i = 0; i < n; i++) {
```

```

if (finish[i] == 0) {
    // Check if Need[i] <= Work
    int canRun = 1;
    for (int j = 0; j < m; j++) {
        if (need[i][j] > work[j]) {
            canRun = 0;
            break;
        }
    }
    // Step 3: Allocate resources and mark finished
    if (canRun) {
        for (int j = 0; j < m; j++)
            work[j] += alloc[i][j];
        finish[i] = 1; // true
        safeSeq[count++] = i;
        found = 1;
    }
}
if (!found)
    break; // No process found, not safe
}

// Step 4: Check if all finished
if (count == n) {
    printf("\nSystem is in a safe state.\n");
    printf("Safe sequence is: ");
    for (int i = 0; i < n; i++) {
        printf("P%d", safeSeq[i]);
        if (i != n - 1)
            printf(" -> ");
    }
    printf("\n");
} else {
    printf("\nSystem is NOT in a safe state.\n");
}

return 0;
}

```

Output:

```
name:riya gupta
USN:1WA24CS235
Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

Need Matrix:
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1

System is in SAFE State
Safe Sequence:
P1 -> P3 -> P4 -> P0 -> P2
```

Deadlock Detection - Several Instances Banker's Algorithm

```
#include <stdio.h>

int main() {

    int n, m;

    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter the number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], request[n][m], avail[m];
    int finish[n], safeSeq[n];

    // Input Allocation Matrix
    printf("Enter the allocation matrix:\n");
    for (int i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for (int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    }

    // Input Request Matrix
    printf("Enter the request matrix:\n");
    for (int i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for (int j = 0; j < m; j++)
            scanf("%d", &request[i][j]);
    }

    // Input Available Resources
    printf("Enter the available resources: ");
    for (int j = 0; j < m; j++)
        scanf("%d", &avail[j]);

    // Step 1(a): Work = Available
    int work[m];
    for (int j = 0; j < m; j++)
        work[j] = avail[j];

    // Step 1(b): If Allocation[i] != 0, Finish[i] = false, else true
    for (int i = 0; i < n; i++) {
        int hasAlloc = 0;
        for (int j = 0; j < m; j++) {
            if (alloc[i][j] != 0) {
                hasAlloc = 1;
                break;
            }
        }
    }
}
```

```

    }
}
finish[i] = hasAlloc ? 0 : 1; // 0 = false, 1 = true
}

// Step 2 & 3: Detection Algorithm
int count = 0;
while (count < n) {
    int found = 0;
    for (int i = 0; i < n; i++) {
        if (finish[i] == 0) {
            // Check if Request[i] <= Work
            int canRun = 1;
            for (int j = 0; j < m; j++) {
                if (request[i][j] > work[j]) {
                    canRun = 0;
                    break;
                }
            }
            // Step 3: Release resources, mark finished
            if (canRun) {
                for (int j = 0; j < m; j++)
                    work[j] += alloc[i][j];
                finish[i] = 1;
                safeSeq[count++] = i;
                found = 1;
            }
        }
    }
    if (!found)
        break;
}

// Step 4: Check for deadlock
int deadlock = 0;
for (int i = 0; i < n; i++) {
    if (finish[i] == 0) {
        deadlock = 1;
        break;
    }
}

if (!deadlock) {
    printf("System is in safe state.\n");
    printf("Safe Sequence is: ");
    for (int i = 0; i < count; i++)
        printf("P%d ", safeSeq[i]);
    printf("\n");
} else {
    printf("System is NOT in safe state. Deadlock detected!\n");
    printf("Deadlocked processes: ");
    for (int i = 0; i < n; i++)
        if (finish[i] == 0)

```

```
    printf("P%d ", i);  
    printf("\n");  
}  
  
return 0;
```

Output:

```
name:riya gupta  
USN:1WA24CS235  
Enter number of processes: 5  
Enter number of resource types: 3  
  
Enter Allocation Matrix:  
0 1 0  
2 0 0  
3 0 3  
2 1 1  
0 0 2  
  
Enter Request Matrix:  
0 0 0  
2 0 2  
0 0 0  
1 0 0  
0 0 2  
  
Enter Available Resources:  
0 0 0  
  
Execution Sequence:  
P0 -> P2 -> P3 -> P4 -> P1
```

Lab 6:

Memory Allocation Techniques

```
#include <stdio.h>

void firstFit(int blocks[], int m, int processes[], int n) {
    int alloc[n], used[m];
    for (int i = 0; i < m; i++)
        used[i] = 0;
    for (int i = 0; i < n; i++)
        alloc[i] = -1;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (!used[j] && blocks[j] >= processes[i]) {
                alloc[i] = j;
                used[j] = 1;
                break;
            }

    printf("\n--- First Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", i + 1, processes[i]);
        if (alloc[i] != -1)
            printf("%d\n", alloc[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void bestFit(int blocks[], int m, int processes[], int n) {
    int alloc[n], used[m];
    for (int i = 0; i < m; i++)
        used[i] = 0;
    for (int i = 0; i < n; i++)
        alloc[i] = -1;

    for (int i = 0; i < n; i++) {
        int best = -1;
        for (int j = 0; j < m; j++)
            if (!used[j] && blocks[j] >= processes[i])
                if (best == -1 || blocks[j] < blocks[best])
                    best = j;
        if (best != -1) {
            alloc[i] = best;
            used[best] = 1;
        }
    }
}
```

```

printf("\n--- Best Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t", i + 1, processes[i]);
    if (alloc[i] != -1)
        printf("%d\n", alloc[i] + 1);
    else
        printf("Not Allocated\n");
}
}

void worstFit(int blocks[], int m, int processes[], int n) {
    int alloc[n], used[m];
    for (int i = 0; i < m; i++)
        used[i] = 0;
    for (int i = 0; i < n; i++)
        alloc[i] = -1;

    for (int i = 0; i < n; i++) {
        int worst = -1;
        for (int j = 0; j < m; j++)
            if (!used[j] && blocks[j] >= processes[i])
                if (worst == -1 || blocks[j] > blocks[worst])
                    worst = j;
        if (worst != -1) {
            alloc[i] = worst;
            used[worst] = 1;
        }
    }
}

printf("\n--- Worst Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t", i + 1, processes[i]);
    if (alloc[i] != -1)
        printf("%d\n", alloc[i] + 1);
    else
        printf("Not Allocated\n");
}
}

int main() {
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int blocks[m];
    printf("Enter sizes of %d memory blocks:\n", m);
    for (int i = 0; i < m; i++)
        scanf("%d", &blocks[i]);

    printf("Enter number of processes: ");
    scanf("%d", &n);

```

```

int processes[n];
printf("Enter sizes of %d processes:\n", n);
for (int i = 0; i < n; i++)
    scanf("%d", &processes[i]);

firstFit(blocks, m, processes, n);
bestFit(blocks, m, processes, n);
worstFit(blocks, m, processes, n);

return 0;
}

```

Output:

Output

```

usn:1WA24CS235
name:Riya gupta
Enter number of memory blocks: 5
Enter sizes of memory blocks:
400
700
200
300
600
Enter number of processes: 4
Enter sizes of processes:
212
512
312
526

```

```

First Fit Allocation:
Process 1 -> Block 1
Process 2 -> Block 2
Process 3 -> Block 5
Process 4 -> Not Allocated

```

```

Best Fit Allocation:
Process 1 -> Block 4
Process 2 -> Block 5
Process 3 -> Block 1
Process 4 -> Block 2

```

```

Worst Fit Allocation:
Process 1 -> Block 2
Process 2 -> Block 5
Process 3 -> Block 2
Process 4 -> Not Allocated

```

```

=== Code Execution Successful ===

```

LAB 7:

Page Frame Replacement Techniques

```
#include <stdbool.h>
#include <stdio.h>

#define MAX_FRAMES 10
#define MAX_PAGES 50

bool isPresent(int frames[], int n, int page) {
    for (int i = 0; i < n; i++)
        if (frames[i] == page)
            return true;
    return false;
}

// Print summary: hits, misses, percentages
void printSummary(const char *algo, int n, int misses) {
    int hits = n - misses;
    printf("\n[%s Summary]\n", algo);
    printf("Total Accesses : %d\n", n);
    printf("Total Hits    : %d\n", hits);
    printf("Total Misses   : %d\n", misses);
    printf("Hit/Miss Ratio : %.2f%%\n", (hits * 100.0) / misses);
}

// FIFO Page Replacement
void fifo(int pages[], int n, int numFrames) {
    int frames[MAX_FRAMES];
    int misses = 0, pointer = 0;

    for (int i = 0; i < numFrames; i++)
        frames[i] = -1;

    printf("\n--- FIFO ---\n");
    printf("%-5s | Frames\n", "Page");

    for (int i = 0; i < n; i++) {
        printf("%-5d | ", pages[i]);

        if (!isPresent(frames, numFrames, pages[i])) {
            frames[pointer] = pages[i];
            pointer = (pointer + 1) % numFrames;
            misses++;
            for (int j = 0; j < numFrames; j++)
                (frames[j] == -1) ? printf("- ") : printf("%d ", frames[j]);
            printf(" *MISS*");
        } else {
            for (int j = 0; j < numFrames; j++)
```

```

        (frames[j] == -1) ? printf("- ") : printf("%d ", frames[j]);
        printf(" *HIT*");
    }
    printf("\n");
}
printSummary("FIFO", n, misses);
}

```

// LRU Page Replacement

```

void lru(int pages[], int n, int numFrames) {
    int frames[MAX_FRAMES];
    int lastUsed[MAX_FRAMES];
    int misses = 0;

    for (int i = 0; i < numFrames; i++) {
        frames[i] = -1;
        lastUsed[i] = 0;
    }

    printf("\n--- LRU ---\n");
    printf("%-5s | Frames\n", "Page");

    for (int i = 0; i < n; i++) {
        printf("%-5d | ", pages[i]);

        if (!isPresent(frames, numFrames, pages[i])) {
            int lruIdx = 0;
            for (int j = 1; j < numFrames; j++)
                if (lastUsed[j] < lastUsed[lruIdx])
                    lruIdx = j;

            frames[lruIdx] = pages[i];
            lastUsed[lruIdx] = i + 1;
            misses++;

            for (int j = 0; j < numFrames; j++)
                (frames[j] == -1) ? printf("- ") : printf("%d ", frames[j]);
            printf(" *MISS*");
        } else {
            for (int j = 0; j < numFrames; j++)
                if (frames[j] == pages[i]) {
                    lastUsed[j] = i + 1;
                    break;
                }

            for (int j = 0; j < numFrames; j++)
                (frames[j] == -1) ? printf("- ") : printf("%d ", frames[j]);
            printf(" *HIT*");
        }
        printf("\n");
    }
    printSummary("LRU", n, misses);
}

```

```

// Optimal Page Replacement
int findOptimalVictim(int frames[], int numFrames, int pages[], int n,
    int current) {
    int farthest = -1, victimIdx = 0;

    for (int i = 0; i < numFrames; i++) {
        int nextUse = n;
        for (int j = current + 1; j < n; j++) {
            if (pages[j] == frames[i]) {
                nextUse = j;
                break;
            }
        }
        if (nextUse > farthest) {
            farthest = nextUse;
            victimIdx = i;
        }
    }
    return victimIdx;
}

void optimal(int pages[], int n, int numFrames) {
    int frames[MAX_FRAMES];
    int misses = 0, filled = 0;

    for (int i = 0; i < numFrames; i++)
        frames[i] = -1;

    printf("\n--- Optimal ---\n");
    printf("%-5s | Frames\n", "Page");

    for (int i = 0; i < n; i++) {
        printf("%-5d | ", pages[i]);

        if (!isPresent(frames, numFrames, pages[i])) {
            if (filled < numFrames)
                frames[filled++] = pages[i];
            else {
                int victim = findOptimalVictim(frames, numFrames, pages, n, i);
                frames[victim] = pages[i];
            }
            misses++;
        }

        for (int j = 0; j < numFrames; j++)
            (frames[j] == -1) ? printf("- ") : printf("%d ", frames[j]);
        printf(" *MISS*");
    } else {
        for (int j = 0; j < numFrames; j++)
            (frames[j] == -1) ? printf("- ") : printf("%d ", frames[j]);
        printf(" *HIT*");
    }
    printf("\n");
}

```

```
    }  
    printSummary("Optimal", n, misses);  
}  
  
int main() {  
    int pages[MAX_PAGES], n, numFrames;  
  
    printf("Enter number of pages: ");  
    scanf("%d", &n);  
  
    printf("Enter the reference string: ");  
    for (int i = 0; i < n; i++)  
        scanf("%d", &pages[i]);  
  
    printf("Enter number of frames: ");  
    scanf("%d", &numFrames);  
  
    fifo(pages, n, numFrames);  
    lru(pages, n, numFrames);  
    optimal(pages, n, numFrames);  
  
    return 0;  
}
```

Output:

Output

Name: Riya Gupta

USN: 1WA24CS235

Enter number of pages: 12

Enter the reference string: 7

0 1 2 0 3 0 4 2 3 0 3

Enter number of frames: 3

--- FIFO ---

Page	Frames
------	--------

7	7 - - *MISS*
---	--------------

0	7 0 - *MISS*
---	--------------

1	7 0 1 *MISS*
---	--------------

2	2 0 1 *MISS*
---	--------------

0	2 0 1 *HIT*
---	-------------

3	2 3 1 *MISS*
---	--------------

0	2 3 0 *MISS*
---	--------------

4	4 3 0 *MISS*
---	--------------

2	4 2 0 *MISS*
---	--------------

3	4 2 3 *MISS*
---	--------------

0	0 2 3 *MISS*
---	--------------

3	0 2 3 *HIT*
---	-------------

[FIFO Summary]

Total Accesses : 12

Total Hits : 2

Total Misses : 10

Hit/Miss Ratio : 20.00%

Output

--- LRU ---

Page	Frames	
7	7 - -	*MISS*
0	7 0 -	*MISS*
1	7 0 1	*MISS*
2	2 0 1	*MISS*
0	2 0 1	*HIT*
3	2 0 3	*MISS*
0	2 0 3	*HIT*
4	4 0 3	*MISS*
2	4 0 2	*MISS*
3	4 3 2	*MISS*
0	0 3 2	*MISS*
3	0 3 2	*HIT*

[LRU Summary]

Total Accesses : 12
Total Hits : 3
Total Misses : 9
Hit/Miss Ratio : 33.33%

Output

Total Accesses : 12
Total Hits : 3
Total Misses : 9
Hit/Miss Ratio : 33.33%

--- Optimal ---

Page	Frames	
7	7 - -	*MISS*
0	7 0 -	*MISS*
1	7 0 1	*MISS*
2	2 0 1	*MISS*
0	2 0 1	*HIT*
3	2 0 3	*MISS*
0	2 0 3	*HIT*
4	2 4 3	*MISS*
2	2 4 3	*HIT*
3	2 4 3	*HIT*
0	0 4 3	*MISS*
3	0 4 3	*HIT*

[Optimal Summary]

Total Accesses : 12
Total Hits : 5
Total Misses : 7
Hit/Miss Ratio : 71.43%

=== Code Execution Successful ===